

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 8: Vector Databases - Implementation and Integration

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Skilling 8	Date	

Aim

To store and query embeddings in PostgreSQL with pgvector (vector column and the <->, <=>, <#> operators), create HNSW and IVFFlat vector indexes, implement hybrid search that combines a keyword rank with vector similarity, outline the RAG retrieval flow, and compare with Weaviate and Pinecone.

Program

weaviate-pinecone_reference.py is the equivalent client code for Weaviate (self-hosted) and Pinecone (managed). It requires a running Weaviate instance and a Pinecone API key, so it is included as reference and its output is not part of this record.

pgvector_search.py

```
"""
Skilling 8: Vector Databases - Implementation and Integration with pgvector.
Covers: vector column + operators (<->, <=>, <#>), HNSW and IVFFlat indexes,
hybrid search (keyword BM25-style rank + vector similarity) and the RAG retrieval step.

Run: pip install psycopg2-binary pgvector numpy scikit-learn
     python pgvector_search.py
"""
import os

import numpy as np
import psycopg2
from pgvector.psycopg2 import register_vector
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer

DSN = os.getenv("TASKHUB_DSN", "dbname=taskhub_db user=postgres password=postgres host=localhost")
DIM = 8

def section(title):
    print(f"\n{'=' * 72}\n{title}\n{'=' * 72}")

def print_rows(cur, headers):
    rows = cur.fetchall()
    widths = [max(len(h), *(len(str(r[i])) for r in rows)) for i, h in enumerate(headers)]
    print(" | ".join(h.ljust(w) for h, w in zip(headers, widths)))
    print("-+-.join("-" * w for w in widths))
    for r in rows:
        print(" | ".join(str(v).ljust(w) for v, w in zip(r, widths)))

# Task Hub help-centre articles (the knowledge base used for semantic search and RAG)
ARTICLES = [
    ("Reset your password", "Use the forgot password link on the login page to receive a reset email and choose a new password."),
    ("Enable two-factor login", "Turn on two-factor authentication in security settings to protect your account login with a code."),
    ("Session token expired", "If your login token expires you are signed out; sign in again to receive a new access token."),
    ("Create a project", "Click New Project, enter a name and deadline, then invite team members to the project."),
    ("Invite team members", "Open project settings and send an invite email so team members can join the project."),
    ("Assign a task", "Open a task and choose an assignee from the project members list to assign the task."),
    ("Set task due dates", "Each task can have a due date; overdue tasks are highlighted in red on the board."),
]
```

```

("Export tasks to CSV", "Use the export button on the task board to download all tasks as a CSV spreadsheet
file."),
("Dark mode", "Switch the dashboard theme to dark mode from the appearance menu in your profile."),
("Dashboard charts", "The dashboard shows charts of completed tasks per week and hours logged per member."),
("Delete a project", "Only the project owner can delete a project; deleted projects cannot be restored."),
("API rate limits", "The public API allows 100 requests per minute per access token before returning HTTP
429."),
]

# ---- Embedding model (LSA: TF-IDF + SVD). Swap for sentence-transformers in production:
# model = SentenceTransformer("all-MiniLM-L6-v2"); vec = model.encode(text, normalize_embeddings=True)
corpus = [f"{t}. {b}" for t, b in ARTICLES]
tfidf = TfidfVectorizer(stop_words="english")
svd = TruncatedSVD(n_components=DIM, random_state=7)
svd.fit(tfidf.fit_transform(corpus))

def embed(text: str) -> np.ndarray:
    vec = svd.transform(tfidf.transform([text]))[0]
    return (vec / np.linalg.norm(vec)).astype(np.float32)

def main() -> None:
    conn = psycopg2.connect(DSN)
    conn.autocommit = True
    cur = conn.cursor()
    cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
    register_vector(conn)

    # =====
    section("1. Table with a VECTOR column, embeddings generated on write")
    cur.execute("DROP TABLE IF EXISTS help_articles")
    cur.execute(f"""
        CREATE TABLE help_articles (
            id          SERIAL PRIMARY KEY,
            title       TEXT NOT NULL,
            body        TEXT NOT NULL,
            embedding   VECTOR({DIM}) NOT NULL,
            tsv         TSVECTOR GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED
        )""")
    for title, body in ARTICLES:
        cur.execute("INSERT INTO help_articles (title, body, embedding) VALUES (%s, %s, %s)",
            (title, body, embed(f"{title}. {body}")))
    cur.execute("SELECT COUNT(*), vector_dims(MIN(embedding::text)::vector) FROM help_articles")
    count, dims = cur.fetchone()
    print(f"Inserted {count} articles, embedding dimension = {dims}")

    # =====
    query = "I cannot sign in, my login token stopped working"
    q = embed(query)
    section(f"2. The three pgvector operators for the query:\n  '{query}'")
    cur.execute("""
        SELECT title,
            ROUND((embedding <-> %(q)s)::numeric, 4)      AS l2_distance,
            ROUND((embedding <=> %(q)s)::numeric, 4)      AS cosine_distance,
            ROUND(((embedding <#> %(q)s) * -1)::numeric, 4) AS inner_product
        FROM    help_articles
        ORDER   BY embedding <=> %(q)s
        LIMIT   4""", {"q": q})
    print_rows(cur, ["title", "<-> L2", "<=> cosine", "-(<#>) inner product"])
    print("<#> returns the NEGATIVE inner product (so ORDER BY ascending works); it is multiplied by -1 above.")

    # =====
    section("3. Vector indexes: HNSW and IVFFlat on 20,000 vectors")
    rng = np.random.default_rng(1)
    centres = rng.standard_normal((200, DIM)) * 3
    vectors = centres[rng.integers(0, 200, 20_000)] + rng.standard_normal((20_000, DIM))
    cur.execute("DROP TABLE IF EXISTS vector_bench")
    cur.execute(f"CREATE TABLE vector_bench (id SERIAL PRIMARY KEY, embedding VECTOR({DIM}))")
    cur.execute("INSERT INTO vector_bench (embedding) SELECT unnest(%s::vector[])",
        ([v.astype(np.float32) for v in vectors],))
    cur.execute("ANALYZE vector_bench")
    probe = vectors[123].astype(np.float32)
    plan_sql = "EXPLAIN (COSTS OFF) SELECT id FROM vector_bench ORDER BY embedding <=> %s LIMIT 5"

    cur.execute(plan_sql, (probe,))
    print("Without an index:\n " + "\n ".join(r[0] for r in cur.fetchall()))

```

```

cur.execute("CREATE INDEX idx_bench_hnsw ON vector_bench USING hnsw (embedding vector_cosine_ops) "
            "WITH (m = 16, ef_construction = 64)")
cur.execute("SET hnsw.ef_search = 40")
cur.execute(plan_sql, (probe,))
print("With HNSW index:\n " + "\n ".join(r[0] for r in cur.fetchall()))

cur.execute("DROP INDEX idx_bench_hnsw")
cur.execute("CREATE INDEX idx_bench_ivf ON vector_bench USING ivfflat (embedding vector_cosine_ops) "
            "WITH (lists = 100)")
cur.execute("SET ivfflat.probes = 10")
cur.execute(plan_sql, (probe,))
print("With IVFFlat index (lists=100, probes=10):\n " + "\n ".join(r[0] for r in cur.fetchall()))

cur.execute("SET enable_indexscan = off")
cur.execute("SELECT id FROM vector_bench ORDER BY embedding <=> %s LIMIT 10", (probe,))
exact_ids = {r[0] for r in cur.fetchall()}
cur.execute("RESET enable_indexscan")
cur.execute("SELECT id FROM vector_bench ORDER BY embedding <=> %s LIMIT 10", (probe,))
ivf_ids = {r[0] for r in cur.fetchall()}
print(f"IVFFlat recall@10 versus exact search: {len(exact_ids & ivf_ids) / 10:.1f}")

# =====
hybrid_query = "export csv"
section(f"4. Hybrid search: keyword rank (ts_rank_cd) + vector similarity for '{hybrid_query}'")
cur.execute("""
    WITH keyword AS (
        SELECT id, ts_rank_cd(tsv, websearch_to_tsquery('english', %(text)s)) AS kw_score
        FROM   help_articles
    ), semantic AS (
        SELECT id, 1 - (embedding <=> %(vec)s) AS vec_score FROM help_articles
    )
    SELECT a.title,
           ROUND(k.kw_score::numeric, 3) AS keyword,
           ROUND(s.vec_score::numeric, 3) AS vector,
           ROUND((0.5 * LEAST(k.kw_score / NULLIF(MAX(k.kw_score) OVER (), 0), 1)
                + 0.5 * s.vec_score)::numeric, 3) AS hybrid
    FROM   help_articles a
    JOIN   keyword   k ON k.id = a.id
    JOIN   semantic  s ON s.id = a.id
    ORDER BY hybrid DESC NULLS LAST
    LIMIT  3""", {"text": hybrid_query, "vec": embed(hybrid_query)})
print_rows(cur, ["title", "keyword", "vector", "hybrid (0.5/0.5)"])

# =====
rag_question = "How do I get back into my account after forgetting the password?"
section("5. RAG: retrieve context -> build prompt -> send to an LLM")
cur.execute("SELECT title, body FROM help_articles ORDER BY embedding <=> %s LIMIT 2",
            (embed(rag_question),))
context = cur.fetchall()
prompt = "Answer the question using only the context below.\n\nContext:\n"
prompt += "\n".join(f"- {t}: {b}" for t, b in context)
prompt += f"\n\nQuestion: {rag_question}\nAnswer:"
print(prompt)
print("\n(The prompt above is the input passed to the LLM, e.g. "
      "client.messages.create(model=..., messages=[{'role': 'user', 'content': prompt}]))")

cur.close()
conn.close()

if __name__ == "__main__":
    main()

```

weaviate_pinecone_reference.py

```

"""
Skillling 8 (Reference): the same semantic search on Weaviate and Pinecone.
Not executed in the lab run: Weaviate needs a running instance
(docker run -p 8080:8080 -p 50051:50051 cr.weaviate.io/semitechnologies/weaviate)
and Pinecone needs a PINECONE_API_KEY.

pip install weaviate-client pinecone
"""
import os

from pgvector_search import ARTICLES, DIM, embed # reuse the same embedding function

# ----- Weaviate (self-hosted) -----
import weaviate

```

```

from weaviate.classes.config import Configure, DataType, Property
from weaviate.classes.query import MetadataQuery

with weaviate.connect_to_local() as client:
    if client.collections.exists("HelpArticle"):
        client.collections.delete("HelpArticle")
    articles = client.collections.create(
        name="HelpArticle",
        vector_config=Configure.Vectors.self_provided(),
        properties=[Property(name="title", data_type=DataType.TEXT),
                    Property(name="body", data_type=DataType.TEXT)],
    )
    with articles.batch.dynamic() as batch:
        for title, body in ARTICLES:
            batch.add_object(properties={"title": title, "body": body},
                             vector=embed(f"{title}. {body}").tolist())

    query_vector = embed("my login token stopped working").tolist()
    result = articles.query.near_vector(near_vector=query_vector, limit=3,
                                       return_metadata=MetadataQuery(distance=True))

    for obj in result.objects:
        print("Weaviate:", obj.properties["title"], round(obj.metadata.distance, 4))

    # Hybrid search = BM25 keyword score fused with vector similarity (alpha 0.5)
    hybrid = articles.query.hybrid(query="export csv", vector=embed("export csv").tolist(),
                                   alpha=0.5, limit=3)

    for obj in hybrid.objects:
        print("Weaviate hybrid:", obj.properties["title"])

# ----- Pinecone (managed cloud) -----
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
if "taskhub-help" not in pc.list_indexes().names():
    pc.create_index(name="taskhub-help", dimension=DIM, metric="cosine",
                   spec=ServerlessSpec(cloud="aws", region="us-east-1"))
index = pc.Index("taskhub-help")

index.upsert(
    vectors=[{"id": f"a{i}", "values": embed(f"{t}. {b}").tolist(),
              "metadata": {"title": t, "category": "account" if "login" in b else "general"}},
            for i, (t, b) in enumerate(ARTICLES)],
    namespace="help-centre",
)
response = index.query(namespace="help-centre", vector=embed("my login token stopped working").tolist(),
                       top_k=3, include_metadata=True,
                       filter={"category": {"$eq": "account"}}) # metadata filtering
for match in response["matches"]:
    print("Pinecone:", match["metadata"]["title"], round(match["score"], 4))

```

Output

python pgvector search.py

```

=====
1. Table with a VECTOR column, embeddings generated on write
=====
Inserted 12 articles, embedding dimension = 8

=====
2. The three pgvector operators for the query:
'I cannot sign in, my login token stopped working'
=====
title | <-> L2 | <=> cosine | -(<#>) inner product
-----+-----+-----+-----
Session token expired | 0.2674 | 0.0358 | 0.9642
API rate limits | 0.7532 | 0.2837 | 0.7163
Enable two-factor login | 1.0353 | 0.5359 | 0.4641
Reset your password | 1.1497 | 0.6608 | 0.3392
<#> returns the NEGATIVE inner product (so ORDER BY ascending works); it is multiplied by -1 above.

=====
3. Vector indexes: HNSW and IVFFlat on 20,000 vectors
=====
Without an index:
Limit
-> Sort
Sort Key: ((embedding <=> '[-2.65099,-0.6486483,0.8924122,-2.8535948,0.27040583,1.3317302,-0.88631284,-1.5438521]':vector))

```

```

-> Seq Scan on vector_bench
With HNSW index:
  Limit
    -> Index Scan using idx_bench_hnsw on vector_bench
        Order By: (embedding <=> '[-2.65099,-0.6486483,0.8924122,-2.8535948,0.27040583,1.3317302,-0.88631284,-1.5438521]':vector)
With IVFFlat index (lists=100, probes=10):
  Limit
    -> Index Scan using idx_bench_ivf on vector_bench
        Order By: (embedding <=> '[-2.65099,-0.6486483,0.8924122,-2.8535948,0.27040583,1.3317302,-0.88631284,-1.5438521]':vector)
IVFFlat recall@10 versus exact search: 1.0

=====
4. Hybrid search: keyword rank (ts_rank_cd) + vector similarity for 'export csv'
=====
title          | keyword | vector | hybrid (0.5/0.5)
-----+-----+-----+-----
Export tasks to CSV | 0.075   | 0.970   | 0.985
Delete a project   | 0.000   | 0.375   | 0.188
Set task due dates | 0.000   | 0.345   | 0.172

=====
5. RAG: retrieve context -> build prompt -> send to an LLM
=====
Answer the question using only the context below.

Context:
- Reset your password: Use the forgot password link on the login page to receive a reset email and choose a new password.
- Enable two-factor login: Turn on two-factor authentication in security settings to protect your account login with a code.

Question: How do I get back into my account after forgetting the password?
Answer:

(The prompt above is the input passed to the LLM, e.g. client.messages.create(model=..., messages=[{'role': 'user', 'content': prompt}])))

```

Result

Twelve help articles were stored with 8-dimensional embeddings. For the login query all three operators ranked 'Session token expired' first. EXPLAIN changed from Seq Scan + Sort to an Index Scan for both the HNSW and IVFFlat indexes, and IVFFlat returned the same top 10 as exact search (recall 1.0). Hybrid scoring placed 'Export tasks to CSV' first, and the RAG step retrieved the password-reset article as context for the LLM prompt.